Deploying Applications

Kubernetes Production · Lesson 8

Built from the official Kubernetes documentation

`kubernetes.io/docs`

Learning Objectives

By the end of this lesson you will be able to:

- Explain why **controllers** are preferred over bare Pods
- Use a **Deployment** for rolling updates, rollbacks and scaling
- Choose between **DaemonSet** and **StatefulSet** for the right workload
- Run batch work with **Job** and **CronJob**
- Use **init containers** and native **sidecar containers**
- Autoscale workloads with the **HorizontalPodAutoscaler** (and know VPA / Cluster Autoscaler)

> Reference: *Workloads*, *Autoscaling Workloads* — kubernetes.io/docs

8.1 Pods vs Controllers

Pods vs Controllers

A **Pod** is the smallest deployable unit, but a bare Pod is **fragile** — it is not rescheduled, scaled or rolled forward. **Workload controllers** wrap a Pod template and continuously reconcile the desired state.

- If a bare Pod's node dies, the Pod is **gone** — nothing recreates it
- Controllers add self-healing, scaling and rollout behaviour

Controller	apiVersion	Manages	Workload type
Deployment	`apps/v1`	ReplicaSet → Pods	Stateless
DaemonSet	`apps/v1`	One Pod per node	Node agents
StatefulSet	`apps/v1`	Pods with identity	Stateful
Job / CronJob	`batch/v1`	Run-to-completion Pods	Batch

When to Use a Bare Pod

Use a standalone Pod **only** for short-lived, throwaway scenarios:

- Quick **debugging** or one-off `kubectl run` testing
- Interactive troubleshooting (`kubectl run -it --rm`)

For anything you care about, use a controller instead:

```
# Throwaway debug Pod — deleted on exit, never recreated
kubectl run tmp --image=busybox:1.28 -it --rm --restart=Never -- sh

# Real workload — managed, self-healing
kubectl create deployment web --image=nginx:1.14.2
```

> CKA tip: in production you almost **never** create a naked Pod — a controller gives you healing, scaling and rollouts for free.

8.2 Deployments

What Is a Deployment?

A **Deployment** provides **declarative** updates for Pods and ReplicaSets. You describe the desired state; the Deployment controller drives actual state toward it at a controlled rate.

- You manage a **Deployment** → it owns a **ReplicaSet** → it owns the **Pods**
- The controller adds a `pod-template-hash` label to each ReplicaSet (auto-managed)
- Best fit for **stateless** applications
- Built-in **rolling updates**, **rollback**, **scaling** and **pause/resume**

> Never manage the underlying ReplicaSet directly — let the Deployment own it.

A Basic Deployment Manifest

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3                # defaults to 1 if omitted
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

> The `selector.matchLabels` **must** match `template.metadata.labels`.

Rolling Updates

A rollout is triggered **if and only if** the Pod template (`.spec.template`) changes — scaling does **not** trigger one. The default strategy is **RollingUpdate**: new Pods come up while old ones drain.

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # extra Pods above replicas (default 25%)
      maxUnavailable: 1    # Pods allowed down (default 25%)
```

- `maxSurge` — how many *extra* Pods may exist during the update
- `maxUnavailable` — how many Pods may be *unavailable* during the update
- `Recreate` strategy kills all old Pods first → **downtime**

Driving & Observing a Rollout

```
# Trigger a rollout by changing the image
kubectl set image deployment/nginx-deployment nginx=nginx:1.16.1

# Watch progress until complete
kubectl rollout status deployment/nginx-deployment

# Pause to batch several edits, then resume as one rollout
kubectl rollout pause deployment/nginx-deployment
kubectl rollout resume deployment/nginx-deployment
```

- `Progressing=True, Available=True` → rollout **complete**
- A stuck rollout (bad image, failing readiness) stays **Progressing**

> CKA tip: `kubectl edit deployment/<name>` also triggers a rollout when it changes the Pod template.

History, Rollback & Scaling

```
# View revision history
kubectl rollout history deployment/nginx-deployment
kubectl rollout history deployment/nginx-deployment --revision=2

# Roll back to the previous (or a specific) revision
kubectl rollout undo deployment/nginx-deployment
kubectl rollout undo deployment/nginx-deployment --to-revision=2

# Scale up or down (no rollout)
kubectl scale deployment/nginx-deployment --replicas=5
```

- `revisionHistoryLimit` (default **10**) caps how many old ReplicaSets are kept
- Old ReplicaSets are retained so you can **roll back**

> CKA tip: `--record` is deprecated; revisions are tracked automatically.

8.3 DaemonSets

DaemonSets — One Pod Per Node

A **DaemonSet** ensures that **all (or some) nodes** run a copy of a Pod.

- A node **joins** → the controller automatically schedules the Pod onto it
- A node is **removed** → its Pod is garbage-collected
- Deleting the DaemonSet **cleans up** all the Pods it created
- There is **no** `replicas` field — count = number of eligible nodes

Typical use cases:

- Cluster **storage** daemon, node **logging** collector (Fluentd), node **monitoring** agent

> The controller auto-adds tolerations (`not-ready`, `unreachable`, pressure, `unschedulable`) so Pods run even on problem nodes.

A DaemonSet Manifest

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-elasticsearch
  namespace: kube-system
  labels:
    k8s-app: fluentd-logging
spec:
  selector:
    matchLabels:
      name: fluentd-elasticsearch
  template:
    metadata:
      labels:
        name: fluentd-elasticsearch
    spec:
      tolerations:
      - key: node-role.kubernetes.io/control-plane
        operator: Exists
        effect: NoSchedule
      containers:
      - name: fluentd-elasticsearch
        image: quay.io/fluentd_elasticsearch/fluentd:v5.0.1
```

> Use `nodeSelector` / `affinity` to target a subset of nodes.

8.4 StatefulSets

StatefulSets — Stable Identity

A **StatefulSet** manages stateful apps where Pods are **not** interchangeable.
Each Pod keeps a **sticky identity** across rescheduling.

Use a StatefulSet when you need:

- **Stable, unique network identifiers** (`web-0`, `web-1`, `web-2`)
- **Stable, persistent storage** that follows each Pod
- **Ordered, graceful** deployment and scaling, plus ordered rolling updates

Behaviour	Order
Create / scale up	`0 → 1 → 2` (one at a time, after Ready)
Delete / scale down	`2 → 1 → 0` (reverse)

Headless Service + StatefulSet

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  clusterIP: None        # Headless — required for stable DNS
  selector:
    app: nginx
  ports:
  - port: 80
    name: web
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "nginx"   # references the headless Service
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: registry.k8s.io/nginx-slim:0.24
```

Stable Storage & DNS

`volumeClaimTemplates` creates **one PVC per Pod**, bound to that Pod's identity:

```
  volumeClaimTemplates:
  - metadata:
      name: www
    spec:
      accessModes: [ "ReadWriteOnce" ]
      resources:
        requests:
          storage: 1Gi
```

- Each Pod gets a stable DNS name via the headless Service:
  `web-0.nginx.default.svc.cluster.local`
- PVCs are **kept** when you scale down or delete the StatefulSet (data safety)

> CKA tip: a StatefulSet **requires** a headless Service (`clusterIP: None`);
> without it, stable network identity does not work.

8.5 Jobs & CronJobs

Jobs and CronJobs

For **batch / run-to-completion** work, not long-running services:

- **Job** — runs Pods until a set number complete **successfully**, then stops
- **CronJob** — creates Jobs on a repeating **schedule** (cron syntax)

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: hello
            image: busybox:1.28
            command: ["sh", "-c", "date; echo Hello from k8s"]
```

> Batch Pods use `restartPolicy: OnFailure` or `Never` — **never** `Always`.

8.6 Init & Sidecar Containers

Init Containers

Init containers run **before** the app containers in a Pod:

- Run **to completion**, **in order** — each must succeed before the next
- **All** init containers must finish before any app container starts
- On failure the kubelet **restarts** them until they succeed
  (unless `restartPolicy: Never` → the Pod fails)
- Do **not** support `livenessProbe`, `readinessProbe`, `startupProbe` or lifecycle hooks

Use cases: wait for a Service, clone a Git repo, render config, run setup scripts kept out of the app image.

Init Container Manifest

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  initContainers:
  - name: init-myservice
    image: busybox:1.28
    command: ['sh', '-c', "until nslookup myservice; do echo waiting; sleep 2; done"]
  containers:
  - name: myapp-container
    image: busybox:1.28
    command: ['sh', '-c', 'echo The app is running! && sleep 3600']
```

```
# Inspect an init container's progress and logs
kubectl describe pod myapp-pod
kubectl logs myapp-pod -c init-myservice
```

Native Sidecar Containers (1.29+)

A **native sidecar** is an **init container with** `restartPolicy: Always`.

- **Enabled by default since Kubernetes 1.29** (graduated to stable in 1.33)
- Starts during init, then **keeps running** for the Pod's whole lifetime
- Terminates **after** the main app containers stop (reverse order)
- **Supports probes** — unlike a plain init container
- Started **before** app containers, so it is ready when the app needs it

> Difference from a regular init container: an init container **stops** before the app starts; a sidecar **stays up** alongside the app.

Sidecar Manifest

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: myapp
        image: alpine:latest
        command: ['sh', '-c', 'while true; do echo hi >> /opt/logs.txt; sleep 1; done']
        volumeMounts:
        - { name: data, mountPath: /opt }
      initContainers:
      - name: logshipper
        image: alpine:latest
        restartPolicy: Always        # makes this a sidecar
        command: ['sh', '-c', 'tail -F /opt/logs.txt']
        volumeMounts:
        - { name: data, mountPath: /opt }
      volumes:
      - name: data
        emptyDir: {}
```

8.7 Workload Autoscaling

From Manual Scaling to Autoscaling

You already scale a Deployment **manually** with `kubectl scale`. A **HorizontalPodAutoscaler (HPA)** does it **automatically** based on observed load.

- HPA adjusts the **replica count** of a Deployment / ReplicaSet / StatefulSet
- Scales on **CPU**, **memory**, or **custom / external** metrics
- **Requires the metrics-server** (or a metrics adapter) to read usage
- Pods **must declare resource** `requests` — utilization is measured against them
- Does **not** apply to **DaemonSets** (their count tracks nodes)

> HPA scales **horizontally** (more Pods). It is itself a control loop that reconciles the current metric toward the desired replica count.

HorizontalPodAutoscaler

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

```bash
# Imperative equivalent + watch
kubectl autoscale deployment php-apache --cpu-percent=50 --min=1 --max=10
kubectl get hpa     # TARGETS = current%/target%, plus REPLICAS
```

```
desiredReplicas = ceil( currentReplicas × currentMetric / targetMetric )
```

VPA & Cluster Autoscaler

Two complementary autoscalers — **separate add-ons, not built in**:

- **Vertical Pod Autoscaler (VPA)** — recommends and can set Pod **requests/limits**, then **restarts** Pods to apply (per-Pod sizing)
- **Cluster Autoscaler** — adds/removes **nodes** when Pods can't schedule or nodes sit underused (cloud-provider driven)

Autoscaler	Changes	Scope
HPA	Number of **Pods**	Workload
VPA	Pod **requests/limits**	Workload
Cluster Autoscaler	Number of **nodes**	Cluster

> Don't run HPA and VPA on the **same** CPU/memory metric — they fight.

Key Takeaways

- Prefer **controllers** over bare Pods — they self-heal, scale and roll out
- **Deployment** = stateless apps; owns a ReplicaSet; `rollout status/history/undo`
- Tune rollouts with `maxSurge` and `maxUnavailable` (default 25% each)
- **DaemonSet** = one Pod per node (no `replicas`); **StatefulSet** = stable identity + per-Pod storage + headless Service
- **Job / CronJob** = run-to-completion batch work (`restartPolicy` OnFailure/Never)
- **Init containers** run first to completion; a **sidecar** is an init container with `restartPolicy: Always` (1.29+)
- **HPA** autoscales replicas on metrics (needs metrics-server + Pod `requests`); **VPA** tunes requests, **Cluster Autoscaler** adds nodes

End of Lesson 8

Next: Lesson 9 — Managing Storage

`kubectl get deploy,ds,sts` · `kubectl rollout status` · `kubectl scale` · `kubectl autoscale`